Name: Shryitha Suddala

Hall Ticket: 2303A51637

Batch: 29

Assessment: 14.2

Task 1: Landing Page Design

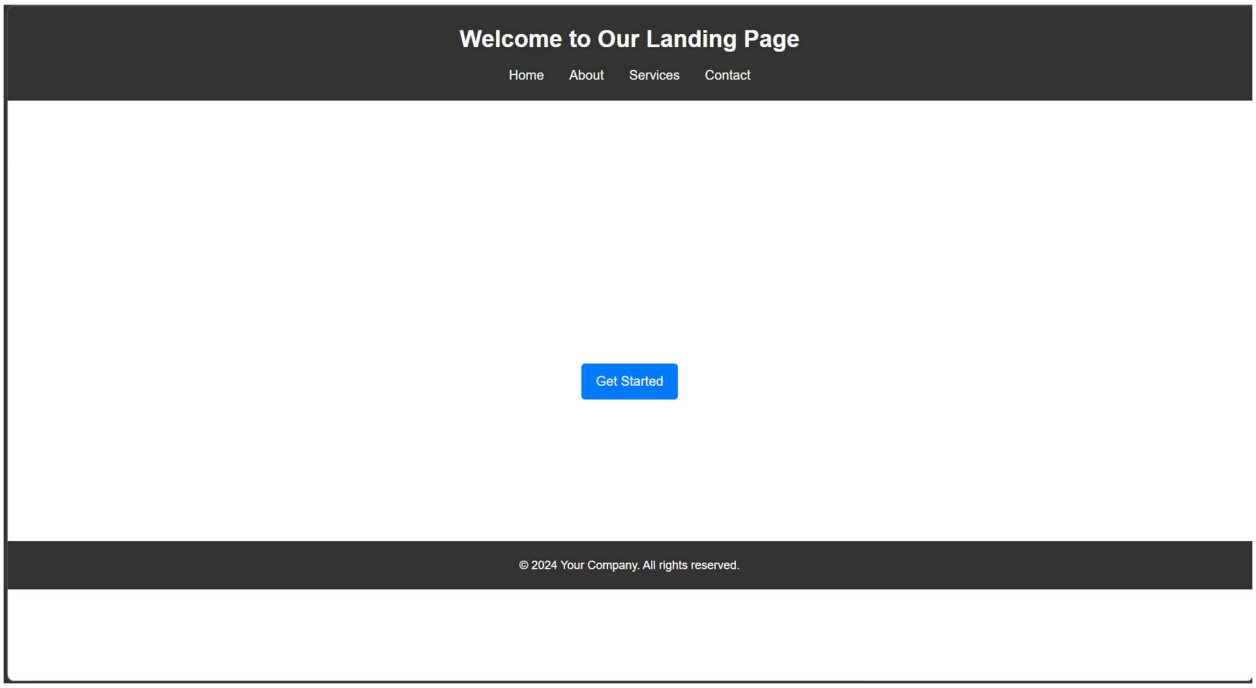
Prompt:

Generate a responsive landing page using HTML and CSS with header, hero section, and footer.

Code:

```html
<!--Generate a responsive landing page using HTML and CSS with header, hero section, and footer.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Landing Page</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: Arial, sans-serif;
            line-height: 1.6;
        }

        /* Header styles */
        header {
            background-color: #333;
            color: #fff;
            padding: 20px 0;
            text-align: center;
        }

        header h1 {
            margin-bottom: 10px;
        }

        nav a {
            color: #fff;
            text-decoration: none;
            margin: 0 15px;
            font-size: 18px;
        }

        nav a:hover {
            text-decoration: underline;
        }
        .hero {
            background-image: url('https://via.placeholder.com/1200x600');
            background-size: cover;
            background-position: center;
            height: 600px;
            display: flex;
            flex-direction: column;
            justify-content: center;
            align-items: center;
            color: #fff;
            text-align: center;
        }

        .hero h2 {
            font-size: 48px;
            margin-bottom: 20px;
        }

        .hero p {
            font-size: 24px;
            margin-bottom: 30px;
```

Output:



Significance:

This task helps in designing a structured and visually appealing web interface using HTML and CSS. It improves understanding of layout, responsiveness, and user experience. It is useful for building modern websites and product pages.

Task 2: Theme Toggle Feature

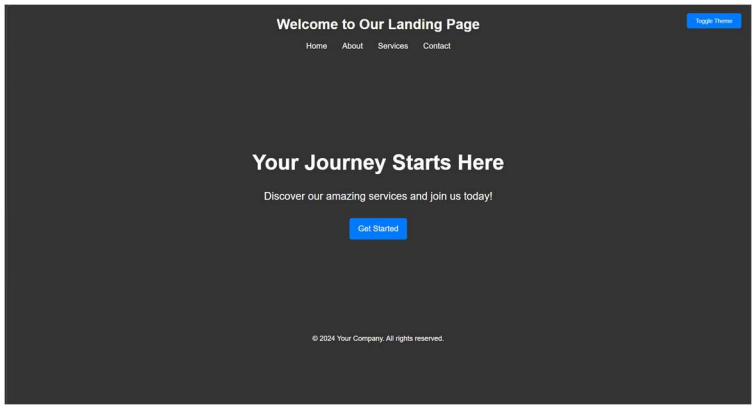
Prompt:

Generate JavaScript code to toggle light and dark theme using CSS classes

Code:

```
<!--Generate JavaScript code to toggle light and dark theme using CSS classes.-->
<script>
    function toggleTheme() {
        const body = document.body;
        body.classList.toggle('dark-theme');
    }
</script>
<style>
    /* Light theme styles */
    body {
        background-color: #fff;
        color: #000;
    }

    /* Dark theme styles */
    .dark-theme {
        background-color: #333;
        color: #fff;
    }

    /* Button styles */
    .theme-toggle-button {
        position: fixed;
        top: 20px;
        right: 20px;
        padding: 10px 20px;
        background-color: #007BFF;
        color: #fff;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    .theme-toggle-button:hover {
        background-color: #0056b3;
    }
</style>
<button class="theme-toggle-button" onclick="toggleTheme()">Toggle Theme</button>
```

Output:



Significance:

This task demonstrates how to enhance user experience by allowing theme customization. It uses JavaScript to dynamically change styles and improve accessibility. It is useful in real-world applications for better usability and user preference.

Task 3: User Feedback Form

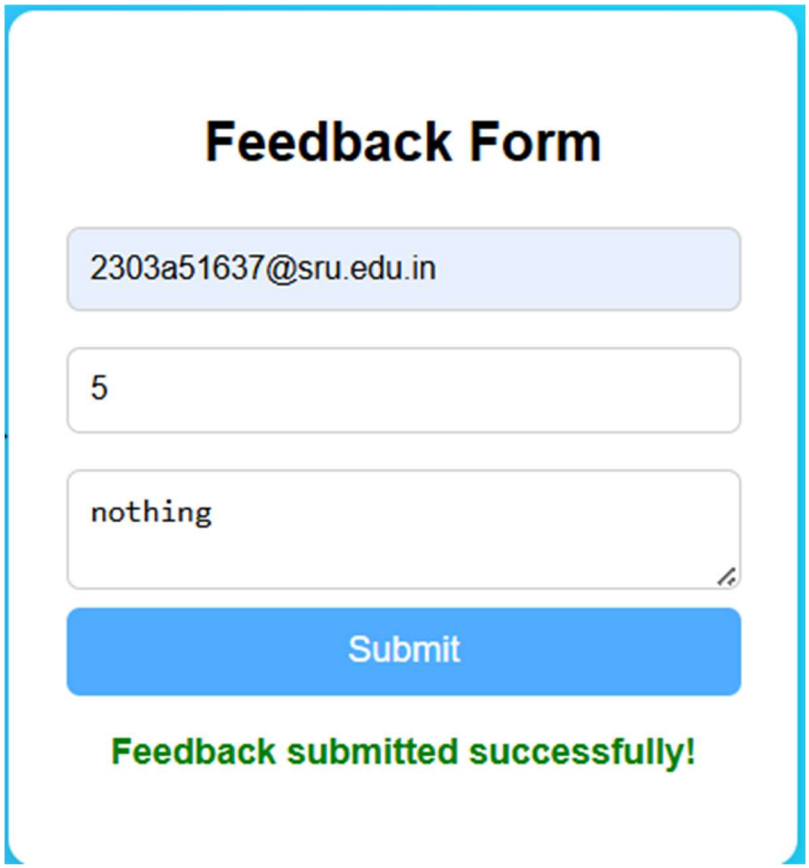
Prompt:

Generate a feedback form with validation using JavaScript and dynamic messages.

Code:

```
<!--Generate a feedback form with validation using JavaScript and dynamic messages.-->
<form id="feedbackForm">
    <h2>Feedback Form</h2>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label for="message">Message:</label>
    <textarea id="message" name="message" required></textarea>

    <button type="submit">Submit</button>
</form>
<div id="feedbackMessage"></div>
<script>
    const feedbackForm = document.getElementById('feedbackForm');
    const feedbackMessage = document.getElementById('feedbackMessage');

    feedbackForm.addEventListener('submit', function(event) {
        event.preventDefault(); // Prevent form submission

        const name = document.getElementById('name').value.trim();
        const email = document.getElementById('email').value.trim();
        const message = document.getElementById('message').value.trim();

        if (name === '' || email === '' || message === '') {
            feedbackMessage.textContent = 'Please fill in all fields.';
            feedbackMessage.style.color = 'red';
            return;
        }

        if (!validateEmail(email)) {
            feedbackMessage.textContent = 'Please enter a valid email address.';
            feedbackMessage.style.color = 'red';
            return;
        }

        feedbackMessage.textContent = 'Thank you for your feedback!';
        feedbackMessage.style.color = 'green';
        feedbackForm.reset(); // Reset the form
    });

    function validateEmail(email) {
        const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        return re.test(String(email).toLowerCase());
    }
</script>
```

Output:



Significance:

This task shows how to collect user input and validate data using JavaScript. It ensures proper form handling and improves user interaction with dynamic messages. It is useful for surveys, reviews, and feedback systems.

Task 4: Image Gallery with JavaScript

Prompt:

Generate an image gallery using JavaScript with next and previous buttons.

Code:

```html
<!--Generate an image gallery using JavaScript with next and previous buttons.-->
<div class="gallery">
    <img id="galleryImage" src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTFYqoKTu_o3Zns2yExbst2Co84Gpc2Q1RJbA&s" alt="Gallery Image">
    <div class="buttons">
        <button id="prevButton">Previous</button>
        <button id="nextButton">Next</button>
    </div>
</div>
<script>
    const images = [
        'https://via.placeholder.com/600x400',
        'https://via.placeholder.com/600x400/ff0000',
        'https://via.placeholder.com/600x400/00ff00',
        'https://via.placeholder.com/600x400/0000ff'
    ];

    let currentIndex = 0;
    const galleryImage = document.getElementById('galleryImage');
    const prevButton = document.getElementById('prevButton');
    const nextButton = document.getElementById('nextButton');
    prevButton.addEventListener('click', function() {
        currentIndex = (currentIndex - 1 + images.length) % images.length;
        galleryImage.src = images[currentIndex];
    });
    nextButton.addEventListener('click', function() {
        currentIndex = (currentIndex + 1) % images.length;
        galleryImage.src = images[currentIndex];
    });
</script>
<style>
    .gallery {
        text-align: center;
        margin-top: 20px;
    }

    .buttons {
        margin-top: 10px;
    }

    .buttons button {
        padding: 10px 20px;
        background-color: #007BFF;
        color: #fff;
        border: none;
        border-radius: 5px;
        cursor: pointer;
        margin: 0 10px;
    }

    .buttons button:hover {
```

Output:

Previous Next

Significance:

This task shows how to dynamically display content using JavaScript. It demonstrates DOM manipulation and interactive UI components. It is useful for building media galleries and portfolio websites

Task 5: Notification Banner

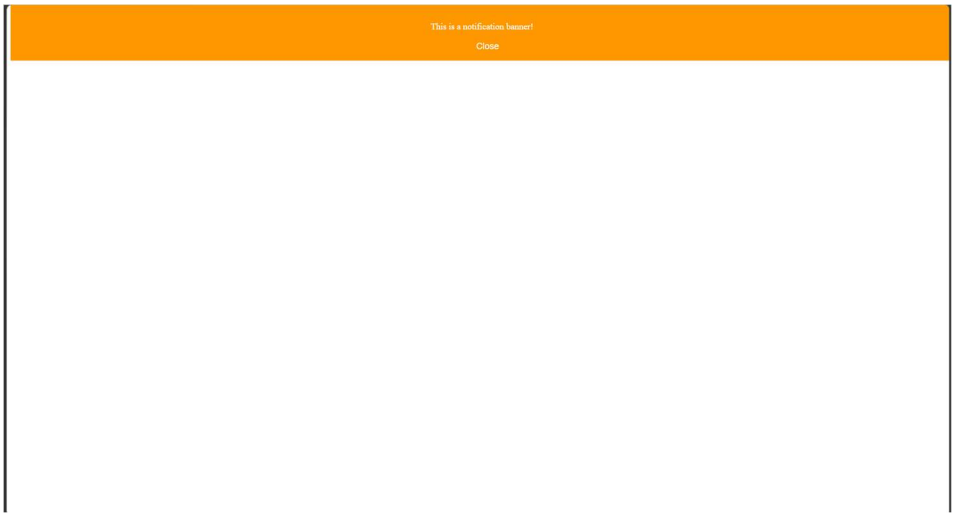
Prompt:

Generate a notification banner using HTML, CSS, and JavaScript with show and hide functionality

Code:

```html
<div id="banner">Welcome User! <button onclick="hide()">X</button></div>

<style>
#banner{
 background: orange;
 padding:10px;
 text-align:center;
}
</style>

<script>
function hide(){
 document.getElementById("banner").style.display="none";
}
</script>
```

Output:



Significance:

This task demonstrates how to display important messages dynamically on a webpage. It shows JavaScript-based visibility control and smooth UI behavior. It is useful for alerts, announcements, and user notifications.